Subgraphs in LangGraph

Build Modular Workflows: Invoke from a Node vs Add as a Node

As a LangGraph workflow grows, one flat graph becomes hard to read, test and change. **Subgraphs** solve this by letting you build a small graph once and use it as a single step inside a bigger graph, like a function inside a program.

This guide explains what subgraphs are, why you need them, and the two ways to connect one to a parent graph. Both ways are built step by step with a real example from your notebooks: a parent graph answers a question in English, and a subgraph translates that answer into Hindi using a local model (`qwen2.5` through Ollama).

What you will learn

- What a subgraph is and how it differs from an ordinary node.
- Why complex agents need subgraphs: modularity, reusability, maintainability, failure isolation, state separation and observability.
- Type 1: calling a subgraph from inside a node, when parent and subgraph have **different state**.
- Type 2: adding a subgraph directly as a node, when parent and subgraph **share state**.
- How to choose between the two types, and how to avoid common mistakes.

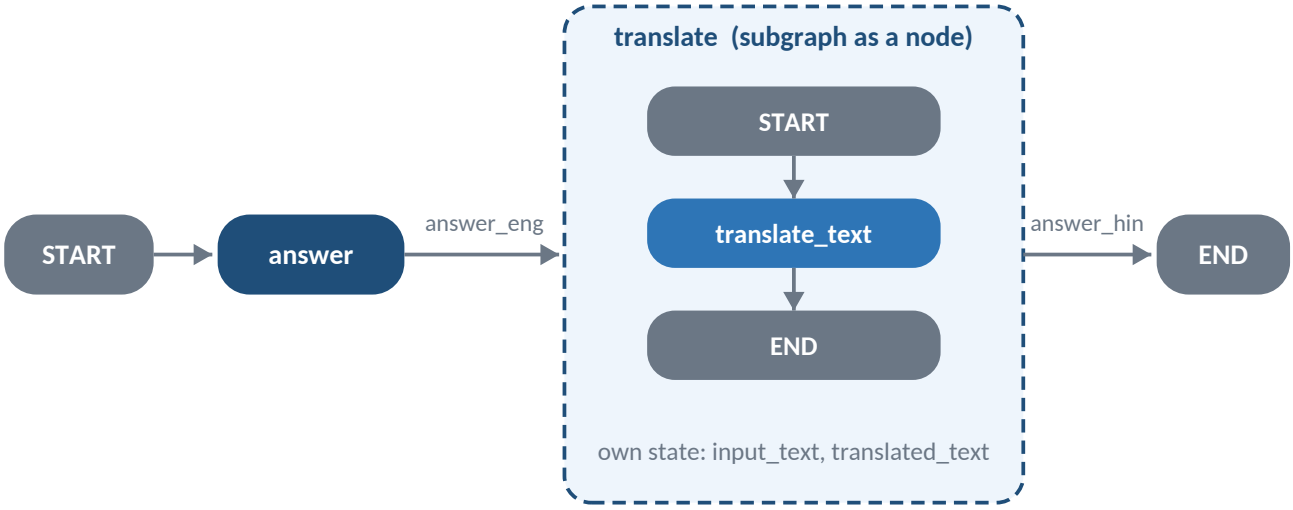
Contents

- 1. What is a subgraph
- 2. Why subgraphs are needed
- 3. The two types of subgraph
- 4. Type 1: invoke a subgraph from a node (different state)
- 5. Type 2: add a subgraph as a node (shared state)
- 6. Choosing between the two types
- 7. Persistence, state inspection and streaming
- 8. Review of the notebooks, mistakes and best practices
- 9. Practice tasks, self-check and cheat sheet

1. What is a subgraph

> **Note:** A subgraph in LangGraph is a graph that is embedded and executed as a node inside another graph.

Every LangGraph graph is built from nodes. Normally a node is a Python function that does one task. With subgraphs, a node can itself be a **whole graph** with its own nodes and edges. The outer graph is called the **parent graph**. The inner one is the **subgraph**.



In the picture above, the parent graph has two steps: `answer` and `translate`. The `translate` step is not a single function. It is a complete small graph with its own START, one node and its own END.

Term	Meaning
Parent graph	The outer graph. It treats the subgraph as one of its nodes
Subgraph	A complete graph that runs inside a node of the parent
Interface	The input and output state keys the subgraph exposes to the parent

The parent graph does not need to know what happens inside the subgraph. As long as the subgraph's input and output (its interface) are respected, you can change the inside freely. This is the same idea as calling a function without reading its code.

2. Why subgraphs are needed

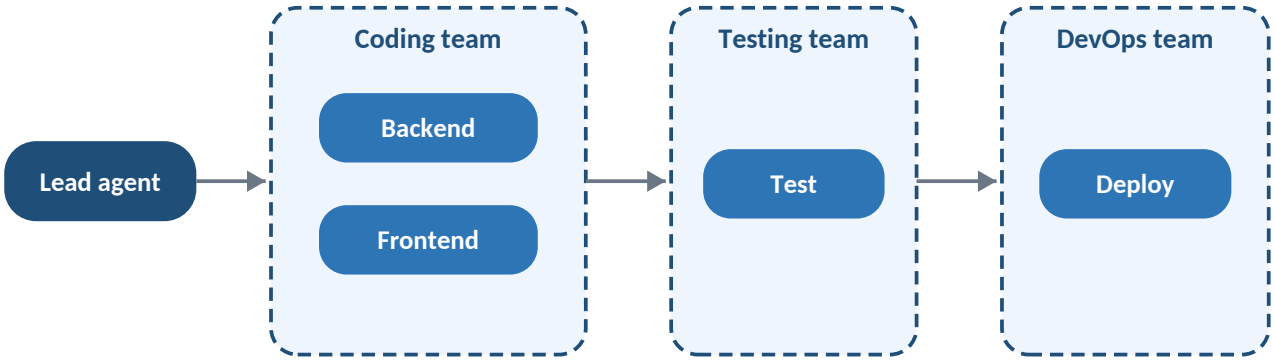
A simple LLM app is a straight line: user, then LLM, then output. Real applications need much more:

Capability	Why it adds complexity
Tool calls	The model chooses tools and loops on their results
RAG	Retrieval, ranking and answer generation are separate steps
Conditional routing	The path changes depending on the state
Retries	Failed steps must be tried again
Memory	State must survive across turns
Human in the loop	The graph pauses for a person to approve or edit
Evaluation and guardrails	Outputs are checked before they are returned

Put all of this in one flat graph and it turns into a tangle of nodes and edges that nobody wants to touch.

Example: a software development agent

Imagine an agent that builds software. A lead agent receives the task. A coding team writes the back end and front end. A testing team checks the result. A DevOps team deploys it. Each team is a group of steps with its own job, so each team can be its own subgraph.



Benefits of subgraphs

Benefit	What it means
Modularity	Split a big workflow into small, understandable pieces
Reusability	Build a subgraph once and use it in many parent graphs
Maintainability	Change one team's graph without touching the others
Failure isolation	A problem inside one subgraph is easier to find and contain
State separation	A subgraph can keep its own private state keys instead of sharing everything
Observability	Traces show which subgraph did what, so debugging is faster

3. The two types of subgraph

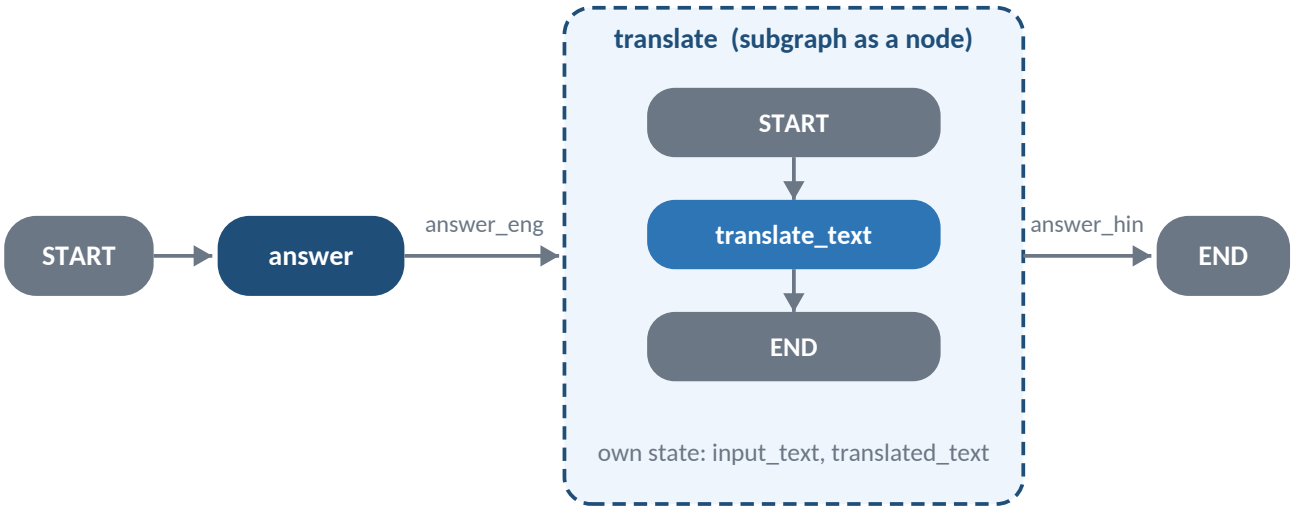
The parent and the subgraph must agree on how to exchange data. LangGraph gives you two ways, and the choice depends on whether the two graphs use the **same state keys** or **different ones**. The official documentation describes them as invoking a graph from a node, and adding a graph as a node.

	Type 1: invoke from a node	Type 2: add as a node
How you connect it	Call `subgraph.invoke(...)` inside a parent node function	Pass the compiled subgraph to `add_node`
State	Different schemas for parent and subgraph	Shared state keys
Data conversion	You write it: parent state in, subgraph result out	None. Shared keys flow automatically
Separation	Strong. The subgraph has private keys	Lighter. Keys are shared with the parent
Your notebook	`15_subgraph.ipynb`	`16_subgraph_shared.ipynb`

Both notebooks solve the same problem, so you can compare them directly. The parent graph takes a `question`, generates an English answer, and the subgraph translates that answer into Hindi.

4. Type 1: invoke a subgraph from a node (different state)

Here the subgraph and the parent graph have **different state schemas**. The subgraph only knows about `input_text` and `translated_text`. The parent knows about `question`, `answer_eng` and `answer_hin`. Since they speak different languages, a parent node acts as a translator between them: it prepares the subgraph's input, calls the subgraph, and copies the result back into the parent's state.



Step 1: build the subgraph with its own state

```
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END
from langchain_ollama import ChatOllama

class subState(TypedDict):
    input_text: str
    translated_text: str

subgraph_llm = ChatOllama(model="qwen2.5")

def translate_text(state: subState):
    prompt = f'''
    Translate the following text into Hindi.
    Keep it natural and clear. Do not add extra content.

    Text:
    {state["input_text"]}'''.strip()
    translated = subgraph_llm.invoke(prompt).content
    return {"translated_text": translated}

subgraph_builder = StateGraph(subState)
subgraph_builder.add_node("translated_text", translate_text)
subgraph_builder.add_edge(START, "translated_text")
subgraph_builder.add_edge("translated_text", END)

subgraph = subgraph_builder.compile()
```

This is a normal graph. It can even be run on its own with `subgraph.invoke({'input_text': '...'})`. That is one of the main benefits: you can test the subgraph separately before connecting it.

Step 2: build the parent graph and call the subgraph from a node

```
class ParentGraph(TypedDict):
    question: str
    answer_eng: str
    answer_hin: str

parentgraph_llm = ChatOllama(model="qwen2.5")

def generate_answer(state: ParentGraph):
    answer = parentgraph_llm.invoke(
        f"You are a helpful assistant. Answer clearly \n\n Question:{state['question']}"
    ).content
    return {"answer_eng": answer}

def translate_answer(state: ParentGraph):
    # 1) convert parent state -> subgraph input
    result = subgraph.invoke({"input_text": state["answer_eng"]})
    # 2) convert subgraph output -> parent state
    return {"answer_hin": result["translated_text"]}

parent_builder = StateGraph(ParentGraph)
parent_builder.add_node("answer", generate_answer)
parent_builder.add_node("translate", translate_answer)

parent_builder.add_edge(START, "answer")
parent_builder.add_edge("answer", "translate")
parent_builder.add_edge("translate", END)

graph = parent_builder.compile()
```

The key function is `translate_answer`. It is an ordinary parent node, but inside it calls `subgraph.invoke`. The two comments mark the two conversions that you must write yourself.

Step 3: run it

```
graph.invoke({"question": "What is quantum Physics?"})
```

The result contains all three parent keys: the question, the English answer (a structured explanation covering energy quantization, wave-particle duality, the uncertainty principle, superposition, entanglement and tunneling) and the Hindi translation.

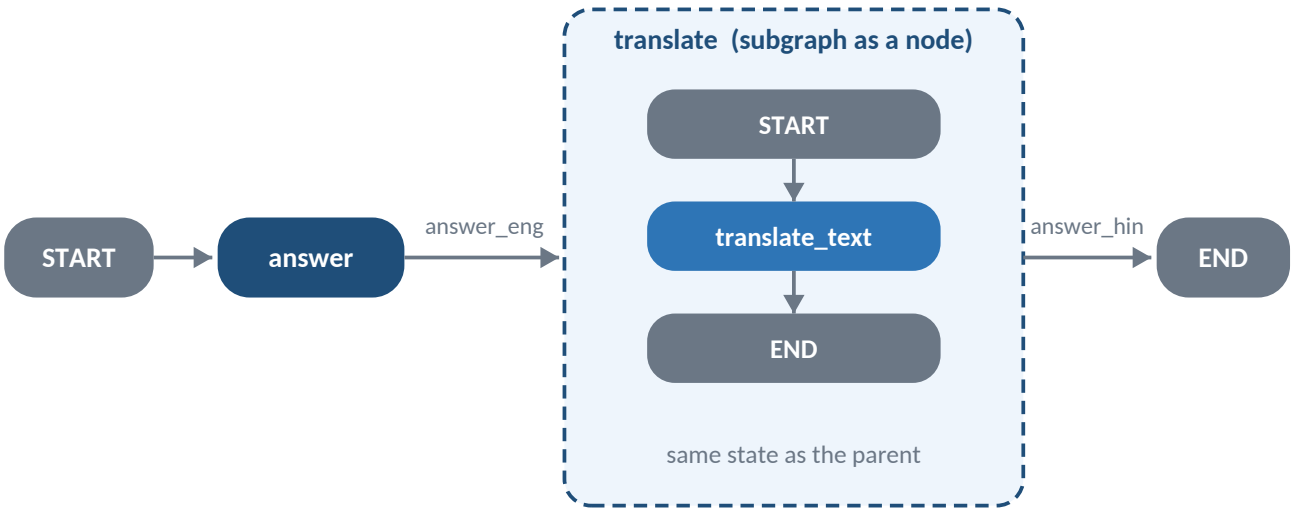
How the state moves

Step	Where	Reads	Writes
1	answer (parent node)	`question`	`answer_eng`
2	translate (parent node)	`answer_eng`	Builds `{'input_text': ...}` and calls the subgraph
3	translated_text (subgraph node)	`input_text`	`translated_text`
4	translate (parent node)	`result['translated_text']`	`answer_hin`

> **Tip:** In Type 1 the subgraph never sees the parent's `question` or `answer_hin`. It only receives what you pass in. That strict separation is the reason to choose this type.

5. Type 2: add a subgraph as a node (shared state)

In the second notebook the subgraph uses the **same state class** as the parent. Because both graphs already share the keys `answer_eng` and `answer_hin`, no conversion code is needed. You add the compiled subgraph directly as a node.



The shared state and the subgraph

```
class ParentState(TypedDict):
    question: str
    answer_eng: str
    answer_hin: str

subgraph_llm = ChatOllama(model="qwen2.5")
parentgraph_llm = ChatOllama(model="qwen2.5")

def translate_text(state: ParentState):
    prompt = f'''
Translate the following text to Hindi.
Keep it natural and clear. Do not add extra content.

Text:
{state["answer_eng"]}
'''.strip()
    translated_text = subgraph_llm.invoke(prompt).content
    return {'answer_hin': translated_text}

subgraph_builder = StateGraph(ParentState)          # same state class as the parent
subgraph_builder.add_node('translate_text', translate_text)
subgraph_builder.add_edge(START, 'translate_text')
subgraph_builder.add_edge('translate_text', END)

subgraph = subgraph_builder.compile()
```

The subgraph node reads `answer_eng` and writes `answer_hin`. Both keys belong to the parent as well.

The parent graph with the subgraph as a node

```
def generate_answer(state: ParentState):
    answer = parentgraph_llm.invoke(
        f"You are a helpful assistant. Answer clearly.\n\nQuestion: {state['question']}"
    ).content
    return {'answer_eng': answer}

parent_builder = StateGraph(ParentState)

parent_builder.add_node("answer", generate_answer)
parent_builder.add_node("translate", subgraph)       # the compiled subgraph IS the node

parent_builder.add_edge(START, 'answer')
parent_builder.add_edge('answer', 'translate')
parent_builder.add_edge('translate', END)

graph = parent_builder.compile()
graph.invoke({'question': 'What is quantum physics'})
```

Compare this with Type 1. The line `add_node("translate", subgraph)` replaces the whole `translate_answer` function. There is no manual input or output mapping. The parent passes its state in, and the keys that the subgraph updates flow back.

Step	Where	Reads	Writes
1	answer (parent node)	`question`	`answer_eng`
2	translate_text (inside the subgraph)	`answer_eng`	`answer_hin`

> **Warning:** With shared state, the parent and the subgraph must agree on key names and types. If you rename `answer_eng` in one place, the other graph breaks. If the subgraph shares no keys with the parent, data cannot flow between them, and that is the situation where you should use Type 1 instead.

6. Choosing between the two types

Question	Use
Do the parent and subgraph already use the same keys?	Type 2: add as a node
Are the schemas different, or must the subgraph stay private?	Type 1: invoke from a node
Do you need to prepare the input or post-process the result?	Type 1: you control both conversions
Do you want the least code?	Type 2
Will several different parents reuse this subgraph with their own state?	Type 1: each parent adapts its own state

A simple rule: start with Type 2 when the graphs naturally share keys. Move to Type 1 when you want the subgraph to be an independent building block with its own input and output.

Side by side

Aspect	15_subgraph (Type 1)	16_subgraph_shared (Type 2)
Subgraph state	`subState`	`ParentState`
Subgraph input	`input_text`	`answer_eng`
Subgraph output	`translated_text`	`answer_hin`
Connection code	`translate_answer()` function	`add_node("translate", subgraph)`
Glue code	A few extra lines	None
Standalone testing	Easy: its input is simple	Needs parent-shaped state

7. Persistence, state inspection and streaming

The LangGraph documentation page on subgraphs also covers three topics that matter once your subgraphs work: adding persistence, viewing the state of a subgraph, and streaming the outputs of a subgraph. They are not used in your two notebooks, so treat the snippet below as a starting point and check the current documentation for exact details.

```
from langgraph.checkpoint.memory import InMemorySaver

# Persistence: give the checkpointer to the PARENT graph when you compile it
graph = parent_builder.compile(checkpointer=InMemorySaver())
config = {"configurable": {"thread_id": "t1"}}

# Stream events from inside the subgraph as well
for chunk in graph.stream({"question": "What is quantum physics"}, config, subgraphs=True):
    print(chunk)

# Inspect the state of a subgraph
graph.get_state(config, subgraphs=True)
```

- **Persistence:** compile the parent with a checkpointer. You do not need to pass one to the subgraph yourself.
- **Stream:** `subgraphs=True` includes the updates that happen inside subgraphs, not only the parent's.
- **View state:** `get_state(..., subgraphs=True)` shows the state of the subgraph, which helps when debugging.

This connects to your earlier work on persistence and on LangSmith: a checkpointer remembers each step, and a trace shows each subgraph as its own part of the run, which is the observability benefit from section 2.

8. Review of the notebooks, mistakes and best practices

What to check in your notebooks

Finding	Suggestion
In notebook 15 the subgraph node is named `translated_text`, the same as a state key	It runs in your version, but it is confusing. Use a distinct name such as `translate_node`
The function `translate_text` assigns its result to a local variable also named `translate_text`	Rename the variable, for example `translated`
The prompt variable is spelled `promt` in notebook 15	Rename it to `prompt` to match notebook 16
The Hindi output has mistakes. In notebook 16, some Hindi words are wrong and Cyrillic letters appear inside one word	A small local model is weak at Hindi. Try a stronger multilingual model and add to the prompt: use only Devanagari script
The notebooks print a urllib3 and LibreSSL warning	It is only a warning from the Python environment and does not affect the graph. A newer Python build removes it

> **Tip:** Always read the model's output, not only the fact that the graph ran. A graph can finish successfully and still produce a poor translation.

Common mistakes

Mistake	How to avoid it
Using Type 2 when the schemas share no keys	Use Type 1 and convert the state in a parent node
Forgetting to map the result back to the parent	In Type 1, return `{"answer_hin": result["translated_text"]}` from the parent node
Passing the subgraph before compiling it	Call `.compile()` on the subgraph first, then use the compiled object
Giving a node the same name as a state key	Use clear, different names for nodes and keys
Renaming a shared key in only one graph	Keep shared keys in one place, for example one shared state class
Giving the checkpointer to the subgraph only	Compile the parent with it

Best practices

- Build and test each subgraph on its own before connecting it.
- Give subgraphs one clear job, such as translating, retrieving or reviewing.
- Name the subgraph's input and output keys clearly, because they are its interface.
- Prefer Type 1 for reusable subgraphs, and Type 2 for tightly related steps.
- Keep prompts for each subgraph inside that subgraph.

9. Practice tasks, self-check and cheat sheet

Practice tasks

- **Task 1:** Run the subgraph from notebook 15 alone with `subgraph.invoke({'input_text': 'Hello'})`.
- **Task 2:** Add a second subgraph that summarizes the English answer in three points, and call it before the translation.
- **Task 3:** Change the target language from Hindi to another language by editing only the subgraph prompt.
- **Task 4:** Rewrite notebook 16 so the subgraph uses its own state, and convert it to Type 1.
- **Task 5:** Add a checkpointer to the parent graph and read the subgraph state with `get_state(..., subgraphs=True)`.
- **Task 6:** Draw your own software agent from section 2 with at least two subgraphs.

Self-check questions

Question	Answer
What is a subgraph?	A graph that runs as a node inside another graph
Name two benefits of subgraphs	For example modularity and reusability, or failure isolation and observability
When do you invoke a subgraph from a node?	When parent and subgraph have different state schemas
When do you add a subgraph as a node?	When they share state keys
What must you write yourself in Type 1?	The conversion of parent state to subgraph input, and of the subgraph result back to the parent
In notebook 16, which keys does the subgraph read and write?	It reads answer_eng and writes answer_hin
Where do you attach a checkpointer?	When compiling the parent graph

Cheat sheet

```
# Type 1: different state -> invoke the subgraph from a parent node
def call_sub(state: ParentState):
    result = subgraph.invoke({"sub_input": state["parent_value"]})
    return {"parent_result": result["sub_output"]}

parent.add_node("call_sub", call_sub)

# Type 2: shared state -> add the compiled subgraph directly as a node
sub = sub_builder.compile()          # sub_builder = StateGraph(ParentState)
parent.add_node("sub", sub)
```

Remember	
Subgraph	A compiled graph used as a node of another graph
Type 1	Different state. Invoke inside a node and convert state yourself
Type 2	Shared state. Add the compiled subgraph with add_node
Why	Modularity, reusability, maintainability, failure isolation, state separation, observability